Q2.0 k-Nearest Neighbors [30 marks]

Use k-NN (from sklearn) to predict `gender` based on student features. First, implement the following helper class for feature transformations.

```
class PerFeatureTransformer:
    def __init__(self):
        """Initializes memory for per-feature transformers."""
        ...
```

```
    def fit(self, df: pd.DataFrame, params: dict[str, str]) -> None:
        """Fits transformers for each feature based on the given
           type.
        Parameters:
            df: The dataframe containing features to be transformed.
            params: A dictionary mapping feature name to
               transformation type,
            e.g., {"GPA": "standard", "major": "ordinal", "program":
                "onehot"}.
        """
        ...

    def transform(self, df: pd.DataFrame) -> np.ndarray:
        """Applies the fitted transformers to the corresponding
           features and returns a NumPy array."""
        ...

    def fit_transform(self, df: pd.DataFrame, params: dict[str, str
       ]) -> np.ndarray:
        """Fits and transforms all features in one step using the
           given transformation parameters."""
        ...
```

Now, implement the following class for predicting gender using KNN.

```
class KNNGederPredictor:
    def __init__(self, student_df: pd.DataFrame, username: str):
        """Initializes the predictor with the full student dataset.
           Use the username for plots."""
        ...

    def train_val_test_split(self, test_size: float = 0.2, val_size:
        float = 0.2, seed: int = 42) -> tuple[pd.DataFrame, pd.
       DataFrame, pd.DataFrame]:
        ...
```

```
def get_feature_matrix_and_labels(self, df: pd.DataFrame,
    features: list[str]) -> tuple[np.ndarray, np.ndarray]:
    """
    Extract selected features and gender labels from the
        DataFrame.
    Applies encoding to categorical variables and normalizes
        numeric features. Do not fit encoders or scalers on test
        data. Only transform using previously fitted ones.
    """
```

```
def get_knn_accuracy_vs_k(self, k_values: list[int], distance:
    str = "euclidean") -> list[float]:
    """Calculates accuracy scores for various k values on the
        validation set."""
    ...

def plot_knn_accuracy_vs_k(self, k_values: list[int], distance:
    str = "euclidean") -> None:
    """Plots accuracy scores against k values on the validation
        set."""
    ...

def get_knn_f1_heatmap(self, k_values: list[int], distances:
    list[str]) -> pd.DataFrame:
    """Returns a dataframe with the f1-score for each
        combination on the validation set"""
    ...

def plot_knn_f1_heatmap(self, f1_scores_df: pd.DataFrame) ->
    None: ...

def get_knn_f1_single_feature_table(self, k_values: list[int],
    features: list[str], distance: str = "euclidean") -> pd.
    DataFrame:
    """Creates a table of F1 scores on the test set using only a
        single feature for prediction."""
    ...
```

Perform the following tasks.

- Train/val/test split the dataset and apply the data transforms. [4]
- What value of k (odd values from 1 to 21) gave the highest accuracy on the validation set with Euclidean distance metric? Justify with a plot. [2]
- Repeat the above for distance metrics like Manhattan and Cosine Similarity. [4]
- Report the validation F-1 score vs k for all the three distance metrics. [4]

- Plot a heatmap: k × distance function vs F-1 score. [4]
- Which distance metric performs better? Why might that be? [2]
- Instead of using all student features, an alternative is to use a single feature for prediction. Create an F-1 score table where rows are various values of k, columns are the single features used. Report values for test set for all the distance metrics. [6]
- Which single feature performed the best? How does it compare with the result using all the features? Why? [4]

Q3.0 Linear Regression with Regularization [30 marks]

You will predict `GPA` using student features. Use a validation set to select the hyperparameters.
Start with a function of the following form:

```
def run_poly_regression(X_train, y_train,
                        X_val, y_val,
                        X_test, y_test,
                        degree=1,
                        regularizer=None,
                        reg_strength=0.0):
    """
    Fit a polynomial regression model with optional regularization.

    Parameters:
        degree (int): Degree of the polynomial to fit
        regularizer (str or None): 'l1', 'l2', or None
        reg_strength (float): Regularization coefficient (alpha)

    Returns:
        dict with train, val, and test MSEs, and learned
           coefficients
    """
```

Perform the following tasks.

- For three setups - no regularization, L1 and L2 regularization, repeat the below steps: [8×3=24]
  - Fit polynomial regression models across degrees 1 to 6 [2]
  - Plot polynomial degree vs MSE (on train and validation sets). Describe the trend you observe as degree increases. [3]
  - For each degree, use val MSE to choose the best regularization strength. [1]
  - Plot regularization strength (log scale) vs val MSE for best degree. [2]